Gossip Protocol in Peer-to-Peer Networks

Project Requirements Specification

Distributed Systems Course

Team Members

Oussama Laaroussi
Kawtar Labzae
Mounia Baddou
Elghali Benjelloun

February 25, 2026

Contents

1 Introduction **2**
1.1 Context 2
1.2 Key Characteristics 2

2 Project Objectives **2**

3 Core Components **2**
3.1 Component 1 – Node Joining & Peer Discovery 2
3.2 Component 2 – Message Modeling 4
3.3 Component 3 – Gossip Dissemination 4
3.4 Component 4 – Edge Probability Update Strategies 5

4 Deliverables **5**

5 Open Design Decisions **5**

6 Summary **6**

7 Architectural Overview – Gossip Network Diagram **6**

1 Introduction

Gossip protocols enable peers in a distributed network to broadcast messages to subscribers of a given topic *without* relying on a centralised broker. Each node maintains a small, partial view of its neighbors and forwards messages redundantly so that information eventually reaches most—or all—interested peers, even as nodes join, leave, or fail.

1.1 Context

In peer-to-peer (P2P) networks, every node typically knows only a handful of neighbors. Gossip protocols embrace this partial knowledge and use **probabilistic forwarding** to disseminate information across the network.

1.2 Key Characteristics

- **Dynamic membership** – Peers may join, leave, change addresses, or drop messages at any time.
- **Redundant dissemination** – Gossip spreads information across overlapping paths, improving *resilience* at the cost of increased network traffic.
- **Probabilistic guarantees** – Delivery is not deterministically guaranteed but reaches high coverage with well-tuned parameters.

2 Project Objectives

The objectives of this project are twofold:

1. **Design and implement** a gossip protocol covering the four core components described in Section 3.
2. **Build a simulation engine** that visualises the behaviour of the protocol under different strategies and network conditions.

The implementation should be designed such that it can also be **deployed on real devices**, not only in simulation.

3 Core Components

The protocol is decomposed into four main components.

3.1 Component 1 – Node Joining & Peer Discovery

This component defines **how a node joins** the peer-to-peer overlay network. Key requirements include:

- A joining mechanism that allows a new node to discover and connect to existing peers.
- Support for multiple joining **strategies** (see examples below).

- Re-discovery: when an edge between two nodes is lost (see Component 3), the affected node must fall back to this component to explore and establish new connections.
- **Neighbor recovery** – Whenever a node loses a neighbor, whether due to *node failure* or *edge cut* (probability dropping to zero), the node is **forced** to initiate a new peer discovery round to replace the lost neighbor and maintain connectivity.
- **Fanout parameter** ($f_{\min}$) – Each node must maintain at least $f_{\min}$ active neighbors at all times. If the neighbor count drops below this threshold, the node immediately triggers peer discovery. This parameter controls the *minimum redundancy* of the gossip overlay.
- **Maximum neighbors** ($f_{\max}$) – Each node should not exceed $f_{\max}$ active neighbors, in order to *limit communication overhead* (forwarding cost grows with the number of neighbors). Incoming connection requests are rejected once this cap is reached, or the least-active edge is replaced.

Known Strategies in Practice

The following are well-known peer discovery strategies used in real-world distributed systems:

1. **Bootstrap / Seed Nodes** – A new node contacts one or more well-known, long-lived nodes whose addresses are hard-coded or published out-of-band. The bootstrap node replies with a list of active peers. Used in *Bitcoin*, *Ethereum*, and most production P2P networks.
2. **Random Walk Discovery** – A joining node sends a discovery request to a known peer; that peer forwards the request to a random neighbor, which in turn forwards it again, for a bounded number of hops (TTL). Nodes encountered along the walk are candidate neighbors. Used in *Gnutella 0.6* and several unstructured overlay networks.
3. **Kademlia DHT Lookup** – Nodes are placed in a structured overlay based on XOR distance between node IDs. A joining node performs iterative lookups to locate its closest neighbors in the ID space. Used in *BitTorrent (Mainline DHT)*, *IPFS*, and *Ethereum's devp2p*.
4. **mDNS / Local Broadcast** – On a local network (LAN), a node sends a multicast DNS query or UDP broadcast to discover peers without any pre-configured addresses. Used in *Apple Bonjour*, *libp2p (mDNS module)*, and various IoT protocols.
5. **Peer Exchange (PEX)** – After an initial connection, peers periodically exchange their neighbor lists with each other, allowing the network view to grow organically. Used in *BitTorrent (PEX extension)* and *Gnutella*.
6. **Tracker-based Discovery** – A centralised or semi-centralised tracker maintains a registry of active peers. New nodes query the tracker to obtain a peer list. While not fully decentralised, it provides fast and reliable bootstrapping. Used in classic *BitTorrent* and many game matchmaking systems.

3.2 Component 2 – Message Modeling

This component specifies the **structure** and **semantics** of the messages exchanged in the network.

- Every message contains:
  - **Content** – the payload of the message.
  - **Sender metadata** – sender ID, timestamp, and a monotonic counter.
  - **Channel (topic)** – the topic to which the message belongs.
- *(Design decision – TBD)* Whether to enforce **total ordering** of messages. Enforcing total order adds extra information to each message and may increase network overhead.
- The message format should be extensible, allowing additional metadata fields depending on the chosen strategy.

3.3 Component 3 – Gossip Dissemination

This is the core gossip engine that governs **how messages propagate** through the overlay network.

Message Reception & Forwarding

1. When a node receives a message, it first checks whether it is **subscribed** to the message's channel.
2. If the node *has not yet seen* the message, it **ingests** (delivers) it locally.
3. The node then **forwards** the message to its neighbors according to the forwarding strategy:
   - Each edge has an associated **forwarding probability**; the message is sent along that edge with the corresponding probability.
   - There is an additional probability that the node **drops** the message entirely (simulating failures/congestion).
4. Each node keeps track of how many times it has **forwarded** a particular message. Once the forwarding count exceeds a configurable **threshold**, the node will permanently ignore that message.

Node & Edge Failure Handling

- Nodes can **fail** with a configurable probability.
- If an edge's forwarding probability drops to **zero**, the two nodes are considered *disconnected*:
  - Each node starts a **back-off timer**; after the timer expires, the edge is dropped and the node triggers the **peer discovery** mechanism (Component 1) to find new neighbors.
  - If the edge probability later rises from zero to a positive value, the nodes reconnect and each node **increases its back-off time** (exponential back-off).

3.4 Component 4 – Edge Probability Update Strategies

This component defines **how the forwarding probabilities on edges evolve** over time. Multiple strategies should be supported:

1. **Usage-based reinforcement** – If an edge is used to successfully send or receive a message, its probability is *increased* (the edge is deemed more reliable). Edges that are not used experience a small *probability decay* over time.
2. **Random fluctuation** – Edge probabilities are subject to random perturbations, modeling the unpredictability of real-world network links.
3. **Random walk model** – Probabilities evolve according to a random walk process, providing a stochastic model of link quality.

4 Deliverables

The project will produce the following deliverables:

1. **Protocol Implementation** – A modular codebase implementing the four components described in Section 3. The code must be structured so that each component can be configured or replaced independently.
2. **Simulation Engine** – A simulation environment that:
   - Instantiates a configurable network topology.
   - Runs the gossip protocol with selectable strategies for each component.
   - Provides **visualisation** of message propagation, edge probability evolution, node joins/leaves, and failure events.
3. **Strategy Comparison Report** – An analysis comparing the behaviour and performance of different strategy combinations (e.g., forwarding strategies, edge update policies) under various network conditions.
4. **Deployment-Ready Design** – Documentation and/or configuration showing that the implementation can be deployed on real devices, not only in simulation mode.
5. **Final Report & Presentation** – A written report documenting the design decisions, implementation details, simulation results, and conclusions, accompanied by a presentation to the course instructors.

5 Open Design Decisions

The following design choices are not yet finalised and will be resolved during the implementation phase:

- Whether to enforce **total message ordering** (see Component 2). The trade-off is between stronger consistency guarantees and additional per-message overhead.
- The specific **joining strategy** (or strategies) to implement in Component 1.
- The exact **parameters** for node failure probability, edge decay rate, back-off timers, and receipt counters.

6 Summary

Table 1: Component overview

#	Component	Responsibility
1	Node Joining & Peer Discovery	How nodes enter and re-enter the overlay
2	Message Modeling	Structure and metadata of messages
3	Gossip Dissemination	Forwarding, failure handling, back-off
4	Edge Probability Updates	Strategies for evolving link quality

7 Architectural Overview – Gossip Network Diagram

Figure 1 provides a high-level illustration of the gossip protocol, showing how the four core components interact within a peer-to-peer overlay network.

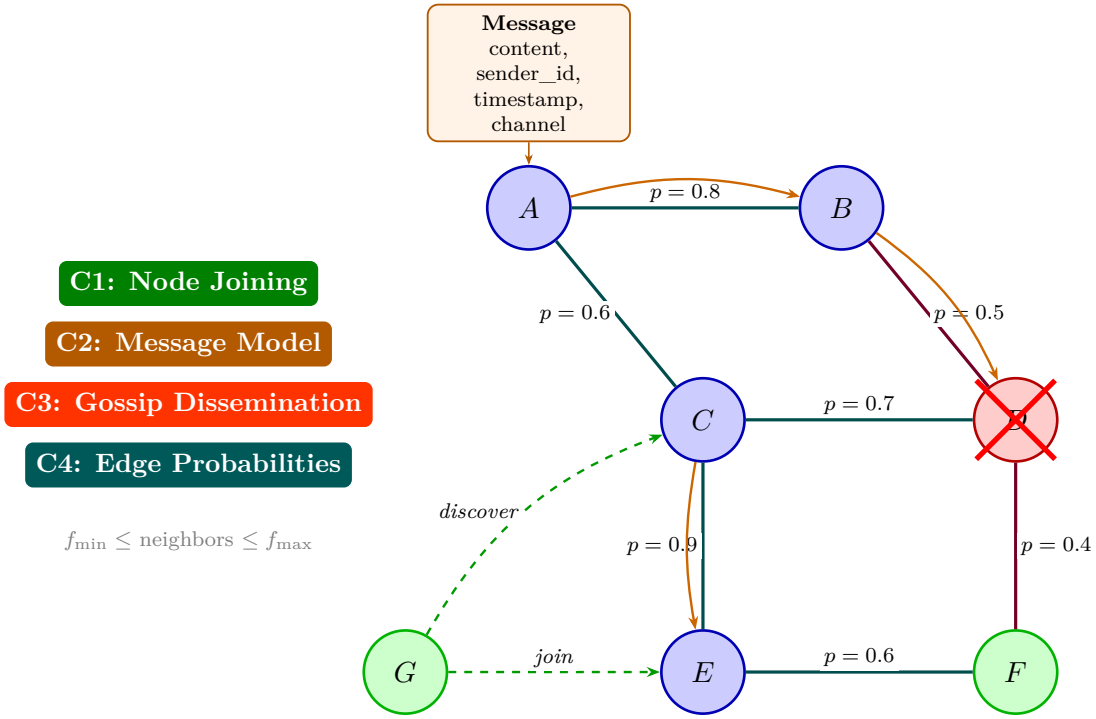


Figure 1: High-level overview of the gossip protocol components. Solid lines show probabilistic edges (C4), wavy arrows show gossip forwarding (C3), dashed arrows show node joining (C1), the message bubble illustrates the message model (C2), and the red cross marks a failed node.